TRABAJO PRÁCTICO INTEGRADOR

Sistema de Gestión de Alquiler de Vehículos

Una empresa dedicada al alquiler de vehículos desea desarrollar una aplicación que permita gestionar sus clientes, vehículos y operaciones de alquiler.

El objetivo del trabajo es desarrollar un sistema utilizando programación estructurada en lenguaje C, incorporando el uso de estructuras de datos, funciones y gestión de archivos binarios y de texto.

El sistema deberá permitir registrar y consultar información de los clientes y vehículos, realizar operaciones de alquiler y devolución, registrar los pagos correspondientes y generar diferentes consultas sobre la información almacenada.

1. FUNCIONALIDADES A IMPLEMENTAR

El sistema deberá contemplar, como mínimo, las siguientes funcionalidades:

REGISTRAR CLIENTE

- Permitir ingresar un nuevo cliente al sistema.
- El sistema deberá validar que el cliente no se encuentre registrado previamente.

REGISTRAR VEHÍCULO

- Permitir incorporar un nuevo vehículo a la flota disponible para alquiler.
- Se deberá registrar la información correspondiente y establecer su estado inicial como disponible.

REALIZAR ALQUILER

Permitir que un cliente realice el alquiler de un vehículo. Para realizar la operación se deberá:

- Verificar que el cliente exista.
- Verificar que el vehículo exista.
- Verificar que el vehículo se encuentre disponible.
- Registrar la fecha de inicio del alquiler.
- Registrar la cantidad de días.
- Calcular el importe total del alquiler.
- Actualizar el estado del vehículo a alquilado.
- Registrar la operación correspondiente.

REGISTRAR DEVOLUCIÓN

- Permitir registrar la devolución de un vehículo. El sistema deberá:
- Identificar el alquiler activo.
- Registrar la fecha de devolución.
- Calcular el importe final, contemplando eventualmente días adicionales.
- Actualizar el estado del vehículo a disponible.
- Registrar la operación.

REGISTRAR PAGO

Permitir registrar el pago total o parcial de un alquiler. El sistema deberá actualizar la información correspondiente y registrar el movimiento realizado.

LISTAR ALQUILERES
Mostrar los alquileres registrados, pudiendo consultar información como:

- Cliente.
- Vehículo.
- Fecha de inicio.
- Fecha de devolución.
- Cantidad de días.
- Importe.
- Estado del alquiler.

CONSULTAR INFORMACIÓN DE UN CLIENTE
Ingresando el DNI del cliente, el sistema deberá mostrar sus datos y el historial de operaciones realizadas.

2. ESTRUCTURAS DE DATOS

El sistema deberá utilizar estructuras (struct) para representar las principales entidades.

CLIENTE
Como mínimo deberá contener:

- DNI
- Apellido y nombre
- Teléfono
- E-mail
- Dirección
- Fecha de alta
- Estado *(Activo o Inactivo)*

El estado podrá indicar, por ejemplo, si el cliente se encuentra activo o inactivo.

VEHÍCULO
Como mínimo deberá contener:

- Patente
- Marca
- Modelo
- Año
- Tipo de vehículo
- Capacidad
- Precio por día
- Estado *(Disponible, Alquilado, Baja o En mantenimiento)*

ALQUILER
Como mínimo deberá contener:

- Número de alquiler
- DNI del cliente
- Patente del vehículo
- Fecha de inicio
- Fecha prevista de devolución
- Fecha efectiva de devolución

- Cantidad de días
- Importe total
- Importe pagado
- Estado *(Activo, Finalizado o Cancelado)*

MOVIMIENTO

Deberá utilizarse una estructura para registrar las operaciones económicas realizadas. Como mínimo deberá contener:

- Número de movimiento
- Número de alquiler
- Tipo de movimiento *(Pago o Ajuste)*
- Monto
- Fecha
- DNI del cliente
- Patente del vehículo

3. GESTIÓN DE ARCHIVOS

El sistema deberá utilizar archivos para mantener la información aun cuando el programa sea cerrado.

Se deberán utilizar, como mínimo, los siguientes archivos binarios:

- **clientes.dat** : Almacenará las estructuras de clientes.
- **vehiculos.dat** : Almacenará las estructuras de vehículos.
- **alquileres.dat** : Almacenará las estructuras de alquileres.
- **movimientos.dat** : Almacenará las estructuras correspondientes a los movimientos.

Además, se deberá generar un archivo de texto para cada cliente:

- **movimientos_DNI.txt** → En dicho archivo se deberá almacenar el historial de movimientos asociados al cliente, utilizando un registro por línea.

4. IMPLEMENTACIÓN

El programa deberá estar organizado mediante funciones, evitando concentrar toda la lógica en la función main(). Como mínimo deberá contemplar funciones para:

- Registrar clientes.
- Modificar clientes.
- Consultar cliente.
- Registrar vehículos.
- Modificar vehículos.
- Buscar vehículo.
- Registrar alquileres.
- Modificar alquileres.
- Registrar movimientos.
- Registrar y Actualizar archivo de movimientos del cliente.
- Registrar devolución.
- Registrar pago.

Cada función deberá tener una responsabilidad claramente definida.

5. MENÚ PRINCIPAL

El sistema deberá disponer de un menú principal que permita acceder a las diferentes funcionalidades. A modo orientativo:

```
=====================================

   SISTEMA DE ALQUILER DE VEHÍCULOS

=====================================

1. Clientes

2. Vehículos

3. Alquileres

4. Devoluciones

5. Pagos

6. Consultas y Reportes

0. Salir


Seleccione una opción:  |
```

El menú podrá organizarse mediante submenús cuando resulte conveniente.

6. CONSULTAS Y REPORTES

El sistema deberá incorporar las siguientes consultas.

VEHÍCULOS DISPONIBLES
Mostrar todos los vehículos que actualmente se encuentran disponibles para alquiler.

VEHÍCULOS ALQUILADOS
Mostrar todos los vehículos que actualmente se encuentran alquilados, indicando:

- Patente.
- Marca y modelo.
- Cliente.
- Fecha de inicio.
- Fecha prevista de devolución.

ALQUILERES POR RANGO DE FECHAS
Solicitar una fecha inicial y una fecha final y mostrar los alquileres realizados dentro de dicho período.

ALQUILERES POR CLIENTE
Ingresando el DNI de un cliente, mostrar todos sus alquileres.

ALQUILERES POR VEHÍCULO
Ingresando la patente de un vehículo, mostrar el historial de alquileres asociados.

ALQUILERES POR ESTADO
Permitir consultar los alquileres según su estado: Activos, Finalizados o Cancelados.

ALQUILERES POR IMPORTE
Solicitar un importe y permitir mostrar:

- Alquileres superiores a dicho importe.
- Alquileres inferiores a dicho importe.

PAGOS REALIZADOS
Mostrar los pagos registrados dentro de un período determinado.

CLIENTES CON DEUDA
Identificar y listar aquellos clientes que posean alquileres cuyo importe total sea superior al importe abonado. La información deberá incluir:

- DNI.
- Nombre.
- Alquiler.
- Importe total.
- Importe pagado.
- Saldo pendiente.

ESTADÍSTICAS DE LA EMPRESA
Generar un resumen que permita conocer:

- Cantidad total de vehículos.
- Cantidad de vehículos disponibles.
- Cantidad de vehículos alquilados.
- Cantidad total de alquileres.
- Cantidad de alquileres activos.
- Importe total cobrado.
- Importe total pendiente de cobro.

7. VALIDACIONES A TENER EN CUENTA

Es fundamental realizar las validaciones necesarias para evitar inconsistencias en la información.

Como mínimo se deberá controlar:

- Que un DNI no se encuentre duplicado.
- Que una patente no se encuentre duplicada.
- Que no se pueda alquilar un vehículo inexistente.
- Que no se pueda alquilar un vehículo que no esté disponible.
- Que no se pueda realizar un alquiler para un cliente inexistente.
- Que las fechas ingresadas sean válidas.
- Que la fecha de devolución no sea anterior a la fecha de inicio.
- Que los importes sean mayores a cero cuando corresponda.
- Que no se registre un pago superior al saldo pendiente.
- Que no se pueda devolver un vehículo que no tenga un alquiler activo.
- Que los archivos puedan abrirse correctamente antes de realizar operaciones sobre ellos.

8. MANEJO DE ERRORES

El programa deberá contemplar situaciones tales como:

- Archivo inexistente.
- Archivo que no puede abrirse.
- Cliente inexistente.
- Vehículo inexistente.
- Alquiler inexistente.
- Datos ingresados incorrectamente. (como ser: tipo, formato, rango, etc.)
- Operaciones no permitidas por el estado actual de los registros.

Los mensajes mostrados al usuario deberán ser claros y permitir comprender cuál fue el problema.

9. REQUISITOS TÉCNICOS

El trabajo deberá desarrollarse respetando los siguientes requisitos:

- Lenguaje de programación: C.
- Aplicación de consola.
- Programación estructurada.
- Utilización de struct.
- Utilización de funciones.
- Utilización de archivos binarios.
- Utilización de archivos de texto.
- Utilización de punteros cuando sean necesarios para el manejo de archivos y estructuras.
- No utilizar librerías o herramientas que automaticen la persistencia de los datos.

La información deberá permanecer disponible luego de cerrar y volver a ejecutar el programa.

10. ORGANIZACIÓN DEL CÓDIGO

Se valorará especialmente que el código se encuentre organizado y modularizado. Se recomienda separar las funcionalidades en diferentes archivos, por ejemplo:

- /main.c
- /clientes.c
- /clientes.h
- /vehiculos.c
- /vehiculos.h
- /alquileres.c
- /alquileres.h
- /movimientos.c
- /movimientos.h

Esta organización será considerada favorablemente en la evaluación.

11. DATOS DE PRUEBA

El grupo deberá entregar los archivos de datos utilizados durante las pruebas. Los datos deberán ser suficientes para demostrar el funcionamiento de todas las funcionalidades solicitadas. Como mínimo se recomienda contar con:

- 10 clientes.
- 10 vehículos.
- 15 alquileres.
- Diferentes estados de vehículos.
- Alquileres activos, cancelados y finalizados.
- Diferentes formas y montos de pago.
- Clientes con y sin deuda.
- Movimientos correspondientes a diferentes fechas.

No será suficiente entregar archivos vacíos o con una cantidad mínima de registros únicamente para demostrar que el programa compila.

12. ENTREGABLES

1. Ejecutable del programa con todas las funcionalidades solicitadas.
2. Código fuente completo.
3. Archivos binarios y de texto utilizados para las pruebas.
4. Carátula con:
   - Universidad.
   - Carrera y Asignatura.
   - Docentes.
   - Integrantes.
   - Año.
5. Breve documentación del sistema, explicando:
   - Funcionalidades implementadas.
   - Estructuras utilizadas.
   - Archivos utilizados.
   - Organización de las funciones.
   - Consideraciones realizadas para las validaciones.